Page No. ( ) Date :

* ASSIGNMENT No. 01 *

SUBJECT: Java Programming

* Submitted by: Alok V. Rahamatkar
* Reg.No : 2024BITO17
* Department: Information Technology (IT)

Printers Nanded

Assignment No. 01

In-Semester Examination - I (Semester II) Solutions:

Que.1] Provide the output exactly as it would appear when executed in the jshell environment.

a. jshell> int a = 'A'; a+4

Ans: a ==> 65

$2 ==> 69

Explaination:

In Java, characters are stored using the Unicode system (which includes ASCII for basic English characters)

In ASCII / Unicode 'A' = 65

for,

int a='A'; // Java Automatically converts the character 'A' into ~~int~~ its integer ASCII value 65

→ That's why jshell shows

a ==> 65

Now for, a+4;

⇒ 65+4

⇒ 69

b. Assume a=5

jshell> -a-++a+--a

Ans: -6

Printers Nanded

Explaination:

Java evaluates expressions left to right

Expression is:

$(-a) - (++a) + (--a)$

part① $\underbrace{(-a)}_{-5}$

$(a=5)$ ---- part②

$a(6)$

$(a=6)$ ------ part③

(5)

$(a=5)$

part1 part2 part3

$\Rightarrow \quad -5 - (6) + (5) \qquad (a=5)$

$\Rightarrow -6$

Precedence order
① postincrement
② preincrement, unary

c. jshell> String b = "A\tb c\sd"

jshell> b.replace(" ","")

Ans: b ==> "A\tb c d"

$6 ==> "A\tbcd"

Explaination: normal space " " & \s have same Unicode.

Hence,

b.replace(" ","") ⟶ replaces the " " (normal space) ASCII/Unicode characters with "" (empty character) Despite replacing \t which have different ASCII code than \s (or " ")

d. jshell> Byte.SIZE + Short.SIZE / Short.Bytes

Ans: Error:

cannot find symbol

symbol: variable Bytes

Byte.SIZE + Short.SIZE / Short.Bytes

Explaination: Java is case-sensitive.

In java wrapper classes like

Byte, Short, Integer, Long

These are pre-defined constants: SIZE, BYTES.

Here,

"Short.Bytes" is syntactically wrong

correction: Byte.SIZE + ShortSIZE / Short.BYTES ⇒ 16

8 + 16/2 = 16.

SIZE returns bits size } Byte → 1 byte

BYTES returns byte size. } Short → 2 bytes

Que. 02: Design and implement a Java method void printPattern (int size) that prints scalable pattern for any valid odd input size. Assume users are passing only positive integer numbers. Your printPattern function must performs following tasks:

1. If 'size' is odd, generates and displays the appropriate X pattern corresponding to the given size.

Printers Nanded

2. If size is 'even', displays the message:
"The entered value <size> is not odd. Please enter an odd size"

Sr. No.	Sample Usage/Input	Sample Output
(1)	printPattern(3)	X X X X X
(2)	printPattern(5)	X X X X X X X X X
(3)	printPattern(4)	The entered value 4 is not odd. Please enter an odd size

Ans:

r↓ \ c→	1	2	3	4	5
1	X				X
2		X		X	
3			X		
4		X		X	
5	X				X

Rows ↓ r

→ c COLUMNS.

row(r)	'X' position (c) (c)
1	(1,5)
2	(2,4)
3	(3,3)
4	(2,4) or (4,2)
5	(1,5) or (5,1)

r == (c || size-c+1)

⇒ (r == c) || (r == size - c + 1)

Program:

```
class Pattern{
    static void printPatter(int size){
        if(size%2==0){
            System.out.println(" The entered value "+
            size +" is not odd. Please enter an odd
            size.");
            return;
        }
        for(int r=1;r<=size;r++){
            for (int c=1; c<=size; c++){
                if(r==c || r==size-c+1){
                    System.out.print("x");
                }
                else{
                    System.out.print(" ");
                }
                System.out.println();
        }
    }
    public static void main(String[] str){
        printPattern(6);
        System.out.print("\n\n");
        printPattern(9);
    }
}
```

Output:

The entered value 6 is not odd. Please enter an odd size.

```
X         X         X
  X     X     X     X
    X   X   X   X
      X X     X X
        X       X
      X X     X X
    X   X   X   X
  X     X     X     X
X         X         X
```

b. Evaluate the implication of explicitly casting large integers to the short type in Java, using (short) 5022026 as an example.

Ans: short → 2 bytes
= 16 bits.

$M = 2^{16} = 65536$

num = 5022026 mod M
= 5022026 mod 65536
= 41290

Printers Nanded

But,

Short range: (-2^{15}) to $(2^{15}-1)$

-32768 to 32767

&

41290 is out of range of -32768 to 32767

So,

$\therefore$ final ans = num - M

$= 41290 - 65536$

$= -24246$

Hence,

Final ans is -24246 for (short) 5022026

c. Can the isBlank() method of the String class always be used as a replacement for isEmpty()? Justify your answer with a suitable example.

Ans:

No, it cannot always be used as a replacement Because, both methods check different conditions.

String.isEmpty() returns true only if length==0

String.isBlank() returns true if the string

* is Empty or
* contains only whitespace characters (spaces, tab, newline, etc)

Printers Nanded

String value	isEmpty()	isBlank()
""	true	true
" "	false	true
"\t"	false	true
"abc"	false	false

Example:

(1) Sometime output is same

```
jshell> String str1="";
str1 ==> ""
jshell> str1.isBlank()
$1 ==> true
jshell> str1.isEmpty()
$2 ==> true
```

(2) Not always same

```
jshell> String str2=" ";
str2 ==> " "
jshell> str2.isBlank()
$3 ==> true
jshell> str2.isEmpty()
$4 ==> false
```

Printers Nanded